IAI600: Introduction to AI & Machine Learning (2.5 Credits)

# Individual Oral Demonstration Master Guide

**Instructor:** Rashid Ali • **Mandatory Individual Examination**
How to show your code running, explain every step, present results, and answer technical questions for **Labs 1, 2, 3, and 5**.

⚠️ **Exact Exam Requirements from Rashid Ali:**

- **Labs Tested:** Labs 1, 2, 3, and 5 ONLY (Labs 4, 6, and 7 are excluded).
- **Core Topics:** Preprocessing pipeline, regression, classification, and clustering.
- **Individual Requirement:** Each student is tested alone. Simply having completed the labs is NOT sufficient; you must demonstrate running code and explain the concepts.
- **Four Requirements:** (1) Show your code running live → (2) Show the execution steps → (3) Present and explain results → (4) Answer technical questions.

## Table of Contents


1. **How the Oral Demonstration Works & Strategy to Pass**
2. **Lab 1: Preprocessing Pipeline** (California Housing • Missing Data • Encoding • Scaling • ColumnTransformer)
3. **Lab 2: Regression** (Linear Regression • Cost Function MSE • Gradient Descent • Polynomial Features • Regularization)
4. **Lab 3: Classification** (MNIST Digits • SGDClassifier • Confusion Matrix • Precision/Recall • ROC-AUC)
5. **Lab 5: Clustering** (Unsupervised Learning • K-Means Algorithm • Centroids • Elbow Method • Silhouette Score)
6. **Teacher's Technical Questions & Exact Answers**


## 1. How the Oral Demonstration Works & Strategy

When your scheduled time slot begins:

1. Have Jupyter Notebook running in your terminal:

```
cd ~/Downloads/handson-ml3-main && conda activate homl3 && jupyter notebook
```

2. Open the corresponding notebook for the lab requested by the teacher:
   - **Lab 1:** `02_end_to_end_machine_learning_project.ipynb`
   - **Lab 2:** `04_training_linear_models.ipynb`
   - **Lab 3:** `03_classification.ipynb`
   - **Lab 5:** `09_unsupervised_learning.ipynb`
3. Select the cell the teacher asks to see, press **Shift + Enter** to show it executing live, and explain the input, process, and output in plain English using the guide below.

## 2. Lab 1: Data Preprocessing Pipeline

**Notebook:** `02_end_to_end_machine_learning_project.ipynb`
**Dataset:** California Housing Dataset (predicting median house value from geographical & demographic features).

### Step 1: Handling Missing Values (Imputation)

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
housing_num = housing.drop("ocean_proximity", axis=1)
imputer.fit(housing_num)
X = imputer.transform(housing_num)
```

- **What this does:** Some houses in the dataset have missing values in `total_bedrooms`. `SimpleImputer` computes the median of the numerical columns on the training set and fills in any missing entries.
- **Why median instead of mean?** The mean is heavily skewed by extreme outliers (very rich mansions). The median is robust against outliers.

### Step 2: Handling Categorical Attributes (One-Hot Encoding)

```
from sklearn.preprocessing import OneHotEncoder
cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing[["ocean_proximity"]])
```

- **What this does:** `ocean_proximity` contains text labels (e.g., "*<1H OCEAN*", "*INLAND*", "*NEAR BAY*"). Machine learning models require numbers.
- **Why One-Hot Encoding?** If we assigned numbers 0, 1, 2, 3, the model would assume that category 2 is "twice as large" or closer to 3 than 0. One-Hot creates binary dummy columns (0 or 1) for each category, eliminating false numerical ordering.

### Step 3: Feature Scaling (StandardScaler)

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
housing_scaled = scaler.fit_transform(housing_num)
```

- **What this does:** Features have completely different scales (e.g., median income ranges from 0 to 15, while total rooms ranges from 6 to 39,000). `StandardScaler` subtracts the mean (μ) and divides by standard deviation (σ), centering features around 0 with unit variance.
- **Why scaling matters:** Algorithms like Gradient Descent, Support Vector Machines, and KNN perform poorly or converge very slowly if features are on vastly different scales.

### Step 4: The Unified ColumnTransformer Pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")),
    ('std_scaler', StandardScaler()),
])

full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", OneHotEncoder(), cat_attribs),
])
housing_prepared = full_pipeline.fit_transform(housing)
```

**Explanation:** `ColumnTransformer` automatically routes numerical columns through the imputation and scaling pipeline, and categorical columns through OneHotEncoder, assembling everything into a single ready-to-train matrix.

---

## 3. Lab 2: Regression

**Notebook:** `04_training_linear_models.ipynb`
**Core Concept:** Predicting continuous target values using linear and polynomial models.

### Step 1: Linear Regression & The Normal Equation

```
from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression()
lin_reg.fit(X, y)
print(lin_reg.intercept_, lin_reg.coef_)
```

- **Mathematical Equation:** $\hat{y} = heta_0 + heta_1 x_1 + \dots + heta_n x_n$.
- **Cost Function:** Mean Squared Error (MSE): $ ext{MSE}( heta) = rac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$.
- Linear Regression finds parameter vector $ heta$ that minimizes this sum of squared errors.

### Step 2: Gradient Descent (Batch vs. Stochastic)

- **Batch Gradient Descent:** Uses the entire training set to calculate gradients at every step. Accurate but slow on massive datasets.
- **Stochastic Gradient Descent (SGD):** Picks a random instance at each step to compute gradients. Extremely fast, but bounces around instead of settling smoothly. Controlled using learning rate decay.

### Step 3: Polynomial Regression & Overfitting

```
from sklearn.preprocessing import PolynomialFeatures
poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)
```

- When data is non-linear, we add powers of each feature (e.g., $x_1^2, x_1 x_2$) to create new features, allowing a linear model to fit curved relationships.

- **Risk:** High degree polynomials (e.g., degree 10 or 300) severely **overfit** — memorizing noise in the training set and failing on new test data.

### Step 4: Regularization (Ridge vs. Lasso)

- **Ridge Regression ($L_2$ Regularization):** Adds $\alpha \sum heta_i^2$ penalty to the cost function. Keeps weights small, preventing any single feature from dominating.
- **Lasso Regression ($L_1$ Regularization):** Adds $\alpha \sum | heta_i|$ penalty. Drives unimportant feature weights completely to **zero**, performing automatic feature selection!

---

## 4. Lab 3: Classification

**Notebook:** `03_classification.ipynb`
**Dataset:** MNIST (70,000 images of handwritten digits 0–9, 28×28 pixels).

### Step 1: Training a Binary Classifier (e.g., "Is this digit a 5?")

```
from sklearn.linear_model import SGDClassifier
y_train_5 = (y_train == '5')
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

### Step 2: The Confusion Matrix

```
from sklearn.metrics import confusion_matrix
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
cm = confusion_matrix(y_train_5, y_train_pred)
```

| | Predicted Negative (Not 5) | Predicted Positive (Is 5) |
|---|---|---|
| **Actual Negative (Not 5)** | **True Negative (TN)** | **False Positive (FP)** (Type I Error) |
| **Actual Positive (Is 5)** | **False Negative (FN)** (Type II Error) | **True Positive (TP)** |

### Step 3: Precision, Recall, and F1-Score

- **Precision:** $\rac{ ext{TP}}{ ext{TP} + ext{FP}}$ — Of all instances predicted positive, what percentage was actually positive?
- **Recall (Sensitivity):** $\rac{ ext{TP}}{ ext{TP} + ext{FN}}$ — Of all actual positives, what percentage did we successfully catch?
- **F1-Score:** $\rac{2 imes ext{Precision} imes ext{Recall}}{ ext{Precision} + ext{Recall}}$ — Harmonic mean balancing both metrics.
- **Precision/Recall Trade-off:** Increasing the decision threshold increases Precision but decreases Recall.

### Step 4: The ROC Curve & ROC-AUC

```
from sklearn.metrics import roc_curve, roc_auc_score
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
auc = roc_auc_score(y_train_5, y_scores)
```

- Plots True Positive Rate vs False Positive Rate. An AUC of 1.0 represents a perfect classifier; 0.5 is random guessing.

---

## 5. Lab 5: Clustering

**Notebook:** `09_unsupervised_learning.ipynb`
**Core Concept:** Unsupervised learning — discovering natural groupings in data without labels.

### Step 1: K-Means Clustering Algorithm

```
from sklearn.cluster import KMeans
k = 5
kmeans = KMeans(n_clusters=k, random_state=42)
y_pred = kmeans.fit_predict(X)
print("Centroids coordinates:", kmeans.cluster_centers_)
```

> 💡 **How K-Means Works Step-by-Step (The Lloyd Algorithm):**
>
> 1. Randomly place $k$ centroids in the data space.
> 2. **Assignment Step:** Assign every data point to its closest centroid (Euclidean distance).
> 3. **Update Step:** Recompute each centroid's location as the mean (average) of all data points assigned to it.
> 4. Repeat steps 2 and 3 until centroids stop moving (convergence).

### Step 2: Inertia and the Elbow Method

```
inertias = []
for k in range(1, 10):
    km = KMeans(n_clusters=k, random_state=42).fit(X)
    inertias.append(km.inertia_)
```

- **Inertia:** The sum of squared distances of samples to their closest cluster center. Lower is tighter.
- **The Elbow Method:** As $k$ increases, inertia always drops. The optimal $k$ is located at the **"elbow"** of the curve — the point where increasing $k$ further yields diminishing returns.

### Step 3: The Silhouette Score

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X, kmeans.labels_)
```

- Measures how close each sample is to other samples in its own cluster compared to samples in neighboring clusters.

- Ranges from -1 to +1:
  - **+1:** Point is well inside its own cluster and far from others (ideal clustering).
  - **0:** Point is on the decision boundary between two clusters.
  - **-1:** Point may have been assigned to the wrong cluster.

---

## 6. Teacher's Technical Questions & Exact Answers

**Q1: "In Lab 1, why do we use fit_transform on training data, but only transform on test data?"**

**Your Answer:** "*Because fitting computes statistics like the mean, median, and variance. If we fit on the test data, our model peeks into the test set, creating 'data leakage'. To evaluate true real-world performance, all scaling and imputation rules must be learned strictly from the training set.*"

**Q2: "In Lab 2, how does Ridge regression prevent overfitting compared to standard Linear Regression?"**

**Your Answer:** "*Standard linear regression minimizes only the mean squared error, which can cause model weights to blow up when fitting noisy data. Ridge regression adds an L2 regularization penalty proportional to the sum of squared weights. This forces the model to keep weights as small as possible, resulting in a flatter, less sensitive curve that generalizes much better.*"

**Q3: "In Lab 3, why is accuracy a misleading metric when classifying rare events, and what do you use instead?"**

**Your Answer:** "*If only 1% of transactions are fraudulent, a dumb classifier that always predicts 'not fraud' will have 99% accuracy while catching zero fraud. On imbalanced data, accuracy is misleading. Instead, we use Precision, Recall, F1-Score, and the Area Under the ROC Curve (ROC-AUC), which evaluate true positive rates independently of class skew.*"

**Q4: "In Lab 5, what is the difference between Inertia and Silhouette Score in finding the optimal k in K-Means?"**

**Your Answer:** "*Inertia only measures how tight points are to their own cluster center, but it will always decrease towards zero as k increases towards the number of data points. Silhouette score is better because it compares the intra-cluster distance against the distance to the nearest neighboring cluster, giving a normalized score from -1 to +1 where the peak indicates true natural separation.*"

**Q5: "In Lab 1, why do we use One-Hot Encoding for nominal categories instead of integer labels (0, 1, 2)?"**

**Your Answer:** "*Integer encoding implies an artificial mathematical order (e.g., category 2 > category 1). Linear models will mistakenly interpret this as a linear relationship. One-Hot Encoding creates independent binary columns for each category, allowing the algorithm to assign dedicated weights without assuming any numerical hierarchy.*"